

3주차 예습과제

■ 과제 종류	예습과제
■ 범위	딥러닝 파이토치 5장

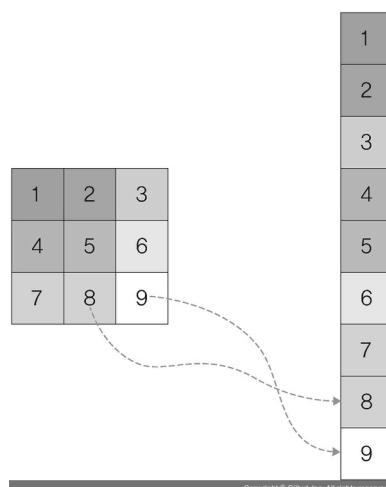
Chapter 05. 합성곱 신경망 I

5.1 합성곱 신경망

합성곱 신경망: 이미지 전체를 한 번에 계산하는 것이 아닌 이미지의 국소적 부분을 계산함으로써 시간과 자원을 절약하여 이미지의 세밀한 부분까지 분석할 수 있는 신경망

5.1.1 합성곱층의 필요성

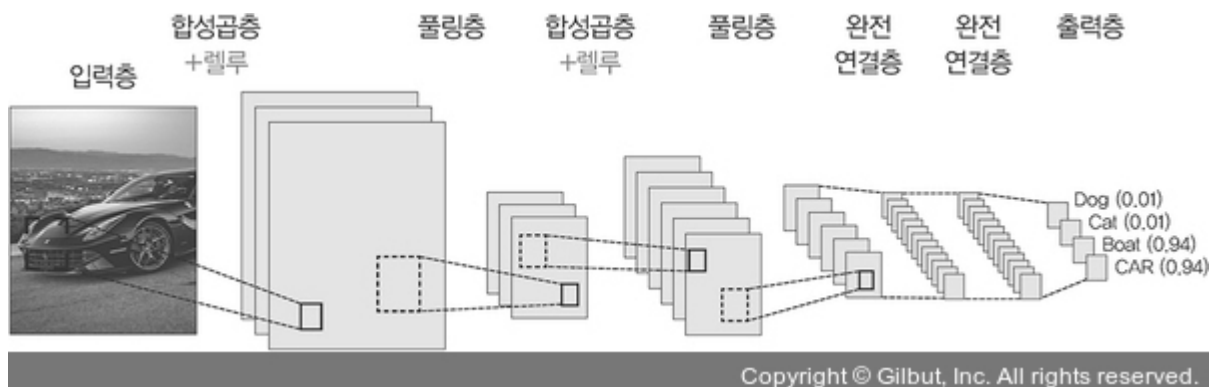
- 합성곱 신경망은 이미지, 영상 처리에 유용
- 이미지 분석
 - 배열을 펼쳐서 각 픽셀에 가중치를 곱한 후, 은닉층으로 전달 (다차원 → 1차원)
 - 펼쳐서 분석하면 데이터의 공간적 구조를 무시하게 되는데, 이것을 방지하려고 도입한 것이 **합성곱층**



합성곱층 원리

5.1.2 합성곱 신경망 구조

- 합성곱 신경망(Convolutional Neural Network, CNN)
 - 음성 인식이나 이미지/영상 인식에서 주로 사용되는 신경망.
 - 다차원 배열 데이터를 처리하도록 구성되어 컬러 이미지 같은 다차원 배열 처리에 특화됨.
- 계층 구조
 - 입력층
 - 합성곱층
 - 풀링층
 - 완전연결층
 - 출력층

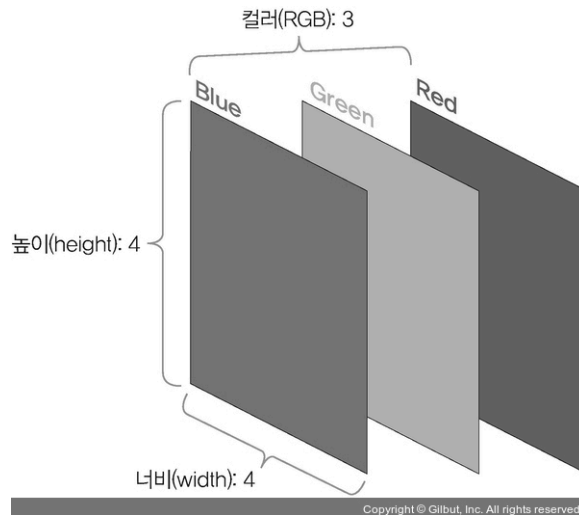


- 과정
 - 합성곱층과 풀링층을 거치면서 입력 이미지의 주요 특성 벡터(feature vector)를 추출
 - 추출된 주요 특성 벡터들은 완전연결층을 거치면서 1차원 벡터로 변환
 - 출력층에서 활성화 함수인 소프트맥스 함수를 사용해 최종 결과 출력

입력층

- 입력층(input layer): 입력 이미지 데이터가 **최초**로 거치게 되는 계층
- 이미지는 단순 1차원 벡터가 아닌, **(높이, 너비, 채널)**의 3차원 텐서 형태로 표현됨.
 - 이때 채널은 이미지 색상 표현 방식에 따라 다른 값을 가짐.

- 그레이스케일(gray scale)이면 → 채널 값: 1
- 컬러(RGB)이면 → 채널 값: 3 (빨강, 초록, 파랑 3개 채널)
- 다음 그림은 높이 4, 너비 4, 채널은 RGB이므로 이미지 형태(shape)는 (4,4,3)으로 표현 가능



합성곱층

- 합성곱층(convolutional layer): 입력 데이터에서 특성을 추출하는 역할
- 특성을 감지하기 위해 커널이나 필터 사용
 - 커널/필터는 이미지의 모든 영역을 훑으면서 특성 추출 → **특성 맵(feature map)**
- 커널은 3x3, 5x5 크기로 적용되는 것이 일반적이며, **스트라이드(stride)**라는 지정된 간격에 따라 순차적으로 이동

스트라이드: 필터를 적용하는 위치의 간격.

- ex) 스트라이드가 2이면 필터를 적용하는 윈도우가 2칸씩 이동한다는 의미

<스트라이드가 1일 때 이동하는 과정>

- (6,6,1)의 크기의 이미지를 3x3 크기의 커널/필터가 스트라이드 1 간격으로 이동하면서 합성곱 연산 수행
- **1단계**: 입력 이미지에 3x3 필터 적용

입력 이미지와 필터를 포개 놓고 대응되는 숫자끼리 곱한 후 모두 더하기

$$(1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) = 3$$



- **2단계** : 필터가 1만큼 이동 후 연산

$$(0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) = 1$$

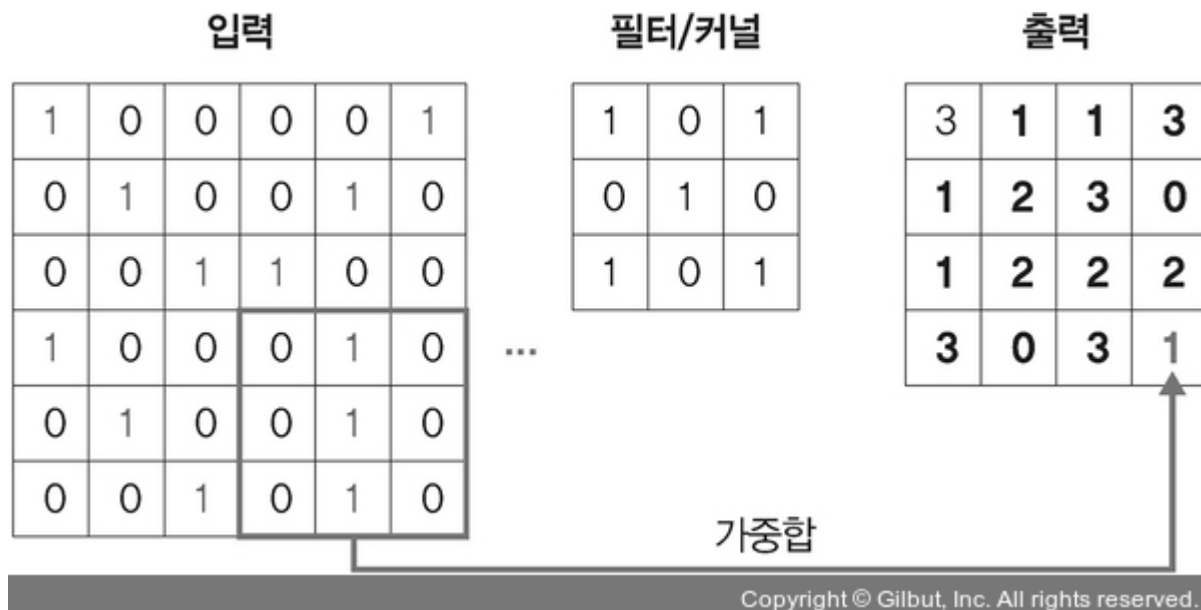


- **3단계** : 필터가 1만큼 두 번째 이동 후 연산

$$(0 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) + (1 \times 0) + (0 \times 1) = 1$$

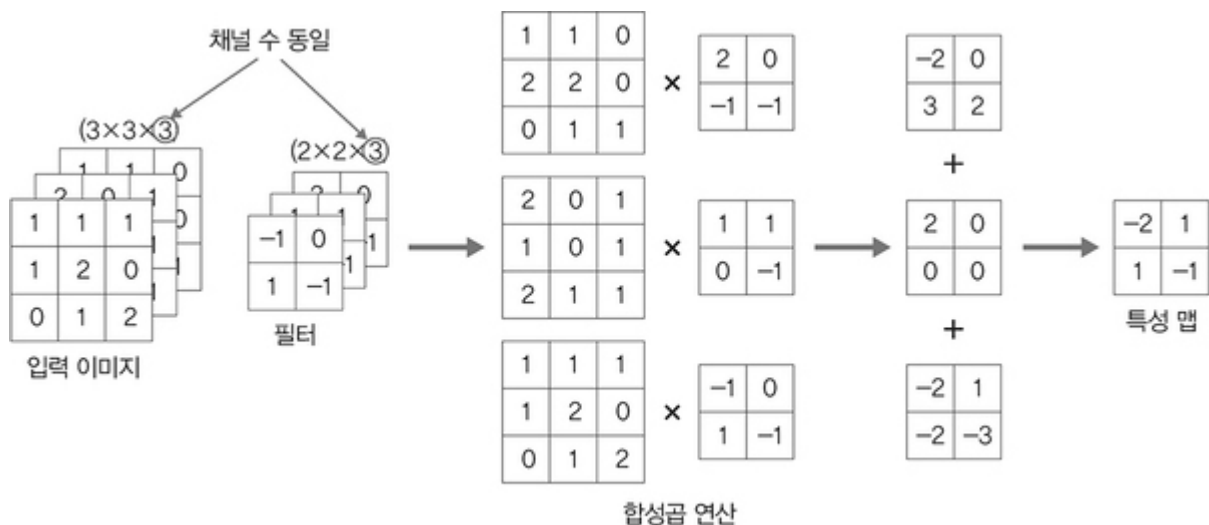


- 이후 단계: 동일하게 연산해서 필터를 1만큼 마지막까지 이동

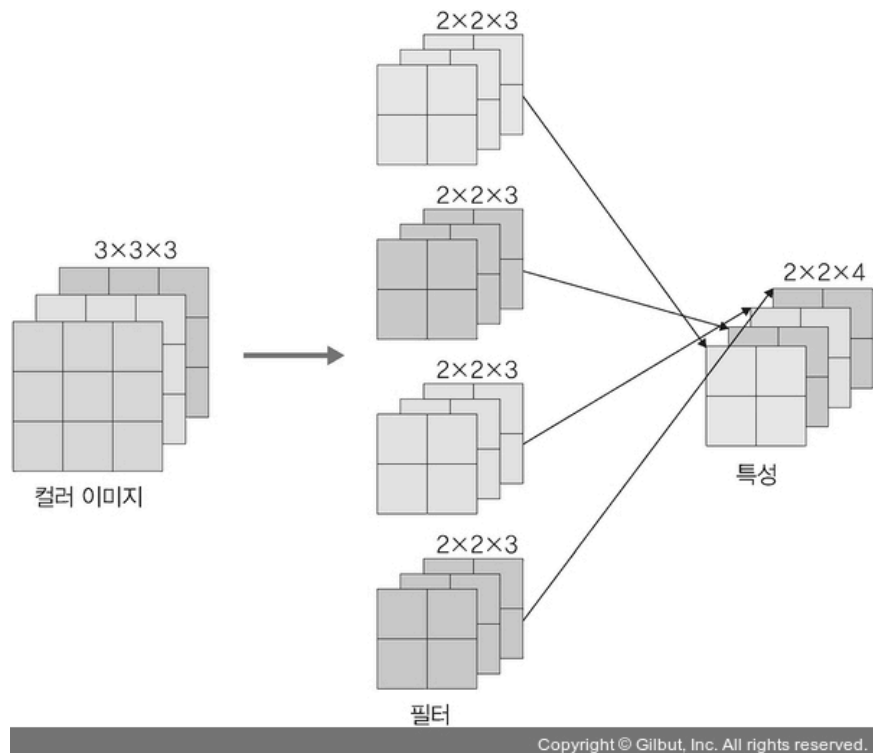


- 커널은 스트라이드 간격만큼 순회하면서 모든 입력 값과의 합성곱 연산으로 새로운 특성 맵을 만들
 → 원본 (6, 6, 1) 크기가 (4, 4, 1) 크기의 특성 맵으로 줄어듦
- 컬러 이미지의 합성곱
 - 그레이스케일 이미지와 구분되는 특징

- 필터 채널이 3
- RGB 각각에 서로 다른 가중치로 합성곱을 적용한 후 결과를 더해줌
- 그 외 스트라이드 및 연산하는 방법은 동일함
- **필터 채널이 3 ≠ 필터가 3개**
 - 필터 한 개가 3개의 채널을 동시에 갖고 있다는 뜻
 - 하나의 필터가 R, G, B 채널 각각에 대해 가중치를 갖고 있고, 이걸 다 합쳐서(feature map) 하나의 출력값을 만든다
- ex)
 - 입력: $32 \times 32 \times 3$ (RGB 이미지)
 - 필터: $5 \times 5 \times 3$
 - R 채널용 5×5 가중치
 - G 채널용 5×5 가중치
 - B 채널용 5×5 가중치
 - 연산:
 1. 각 채널별로 합성곱 연산 수행
 2. 3개의 결과를 모두 더함 (+ bias)
 3. 최종적으로 **하나의 값** → feature map의 한 픽셀



- 필터가 두 개 이상인 합성곱
 - 필터 각각은 특성 추출 결과의 채널이 됨.



- 합성곱층 요약
 - 입력 데이터 : $W1 \times H1 \times D1$ (가로, 세로, 채널 또는 깊이)
 - 하이퍼파라미터
 - 필터 개수 : K
 - 필터 크기 : F
 - 스트라이드 : S
 - 패딩 : P
 - 출력 데이터
 - $W2 = (W1 - F + 2P) / S + 1$
 - $H2 = (H1 - F + 2P) / S + 1$
 - $D2 = K$

풀링층

- 풀링층(Pooling layer): 합성곱층과 유사하게 특성 맵의 차원을 다운 샘플링하여 연산량을 감소시키고, 주요한 특성 벡터를 추출하여 학습을 효과적으로 할 수 있게 한다.

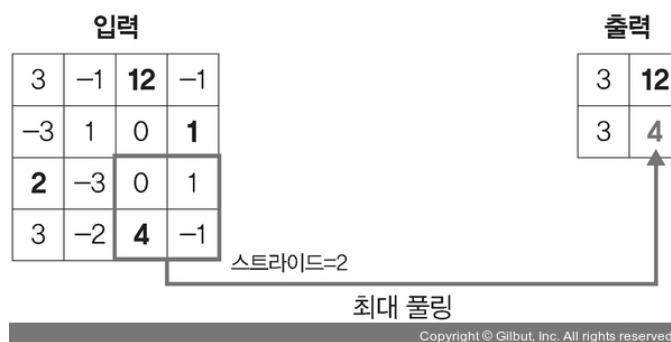
다운 샘플링(sub-sampling): 이미지를 축소하는 것



- 풀링 연산
 - 최대 풀링 (max pooling) : 대상 영역에서 최댓값 추출
 - 평균 풀링 (average pooling) : 대상 영역에서 평균을 반환
- 대부분의 CNN에서는 최대 풀링 사용됨
 - 평균 풀링은 각 커널 값을 평균화시켜 중요한 가중치를 갖는 값의 특성이 희미해질 수 있기 때문

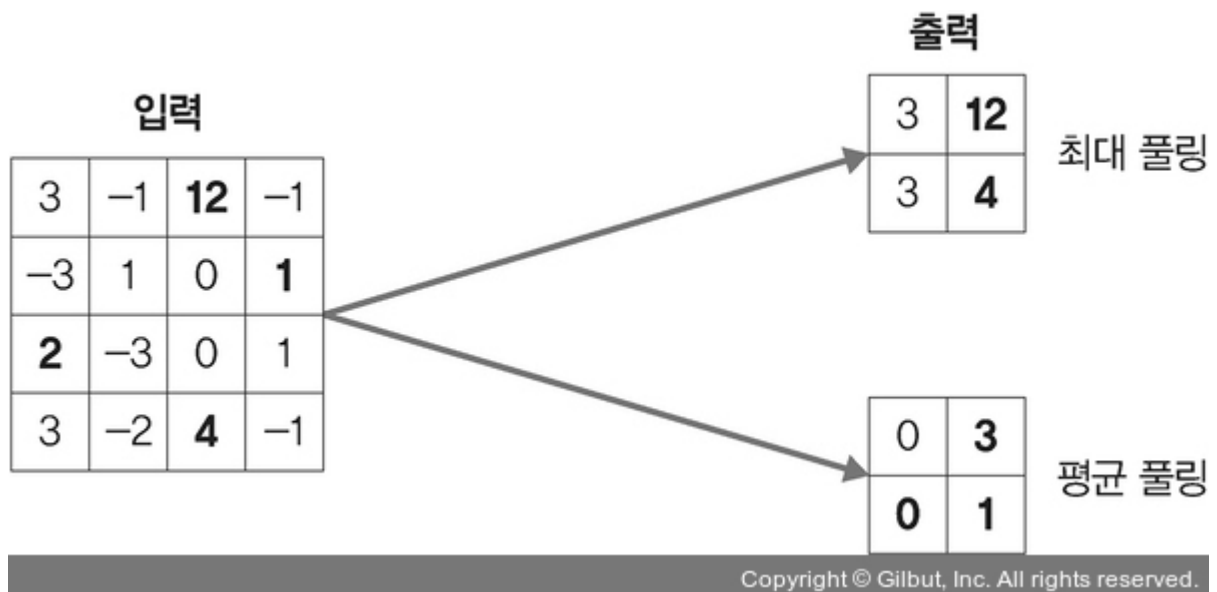
<최대 풀링 연산 과정>

- 주어진 스트라이드를 기준으로 최댓값을 선택한다 → 반복
- ex) 0, 1, 4, -1 값 중에서 최댓값(4)을 선택 (스트라이드=2)



<평균 풀링 연산 과정>

- $0 = (3 + (-1) + (-3) + 1) / 4$
- $3 = (12 + (-1) + 0 + 1) / 4$
- $0 = (2 + (-3) + 3 + (-2)) / 4$
- $1 = (0 + 1 + 4 + (-1)) / 4$

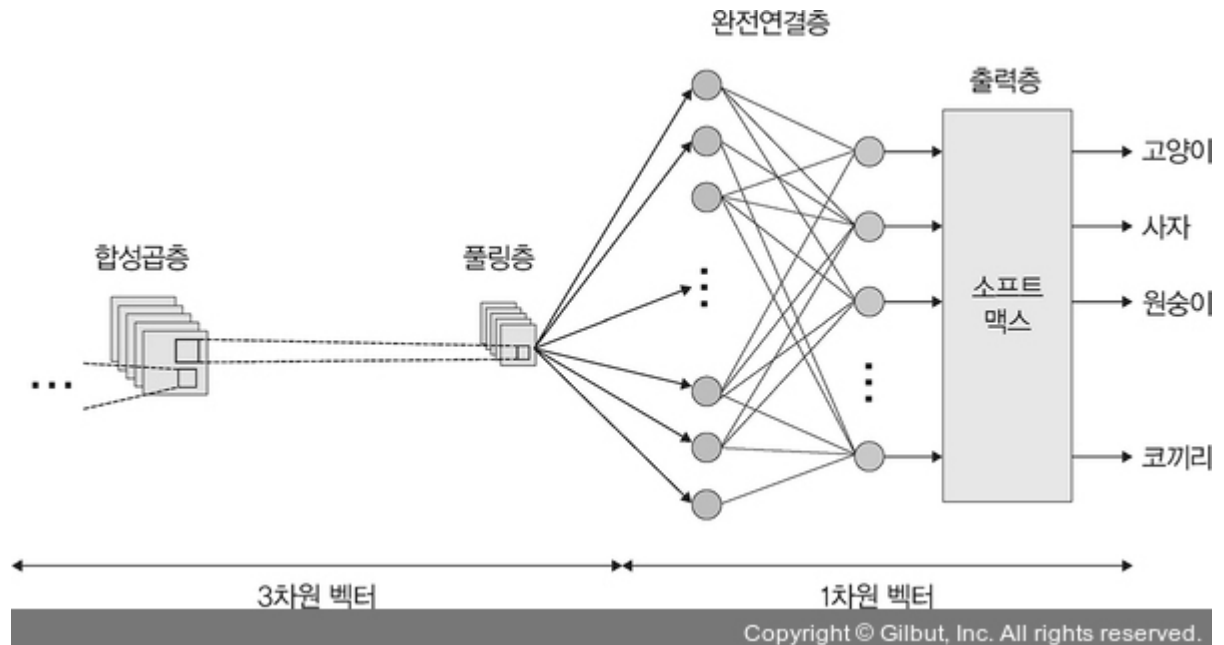


• 최대 풀링, 평균 풀링 요약

- 입력 데이터 : $W1 * H1 * D1$ (가로, 세로, 채널 또는 깊이)
- 하이퍼파라미터
 - 필터 크기 : F
 - 스트라이드 : S
- 출력 데이터
 - $W2 = (W1 - F) / S + 1$
 - $H2 = (H1 - F) / S + 1$
 - $D2 = D1$

완전연결층(fully connected layer)

- 합성곱층과 풀링층을 거치면서 차원이 축소된 feature map은 최종적으로 완전연결층(fully connected layer)으로 전달된다.
- 이미지는 3차원 벡터에서 1차원 벡터로 펼쳐지게(flatten) 됨



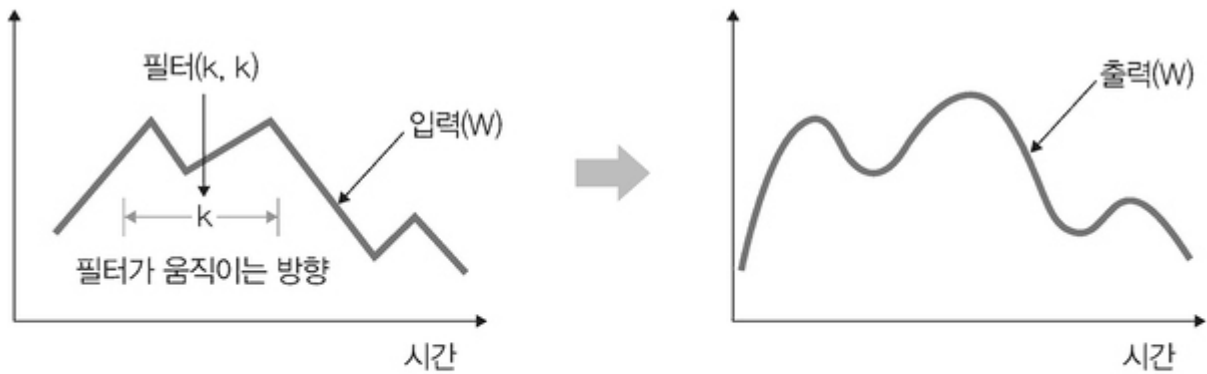
출력층

- 출력층(output layer): 소프트맥스 활성화 함수 사용
- 입력받은 값을 0~1 사이의 값으로 출력한다.
- 이미지가 각 레이블에 속할 확률 값 출력, 이때 가장 높은 확률 값을 갖는 레이블이 최종 값으로 선정

5.1.3 1D, 2D, 3D 합성곱

- 1D 합성곱
 - 필터가 시간을 축으로 좌우로만 이동할 수 있는 합성곱
 - 입력(W)과 필터(k)에 대한 출력은 W가 됨.
 - 입력이 [1,1,1,1,1]이고 필터가 [0.25, 0.5, 0.25]라면 출력은 [1,1,1]
 - 출력 형태는 1D의 배열

- 그래프 곡선을 완화할 때 많이 사용

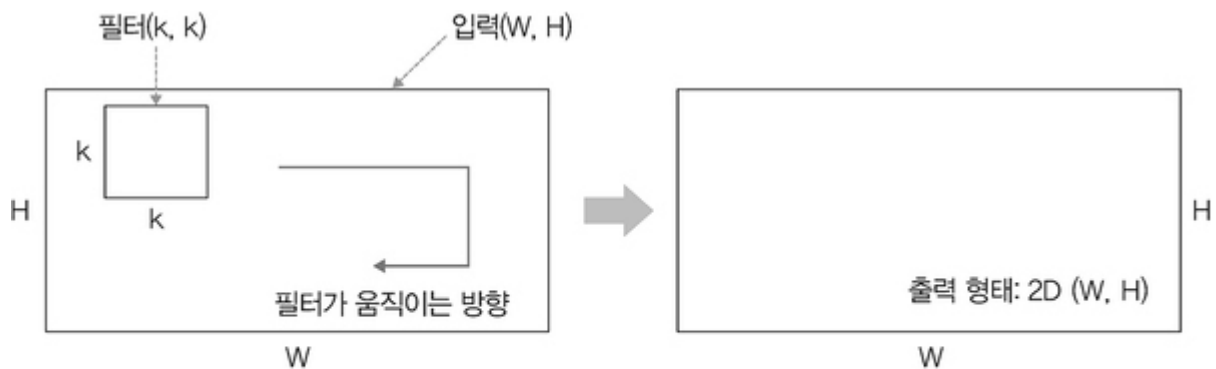


- 입력: W 너비(Width)
- 필터: $k \times k$ (높이 $\times$ 너비)
- 출력: W 너비(Width)

Copyright © Gilbut, Inc. All rights reserved.

• 2D 합성곱

- 방향 두 개로 움직이는 형태
- 입력(W, H)과 필터(k, k)에 대한 출력은 (W, H)가 되며, 출력 형태는 2D 행렬

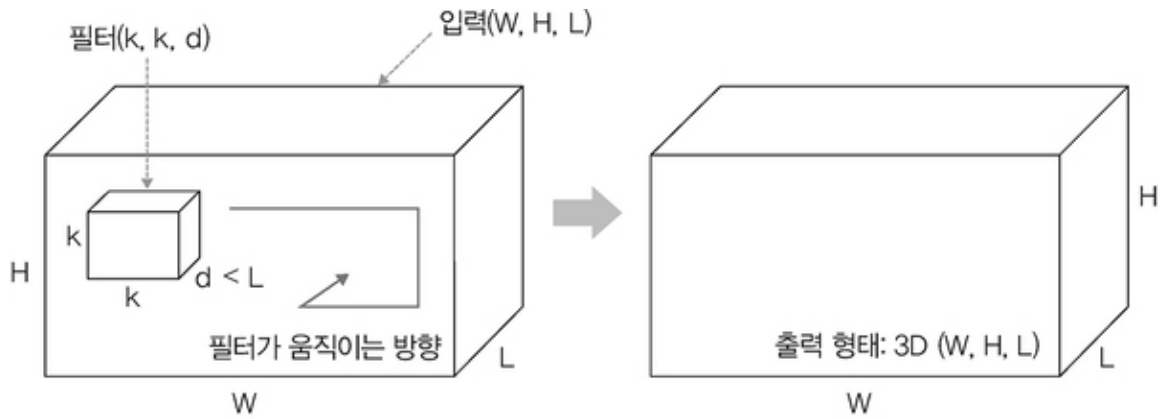


- 입력: W 너비(Width), H 높이(Height)
- 필터: $k \times k$ (높이 $\times$ 너비)
- 출력: W 너비(Width), H 높이(Height)

Copyright © Gilbut, Inc. All rights reserved.

• 3D 합성곱

- 입력(W, H, L)에 대해 필터(k, k, d)를 적용하면 출력으로 (W, H, L)을 갖는 형태
- 출력은 3D 형태이며, 이때 $d < L$ 을 유지하는 것이 중요

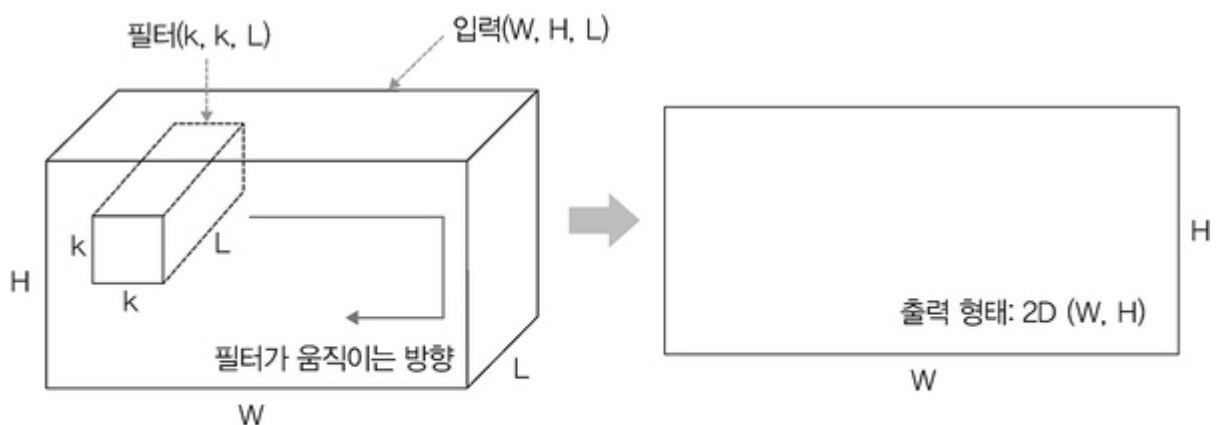


- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: $k \times k$ (높이 $\times$ 너비), d: 깊이(depth)
- 출력: W 너비(Width), H 높이(Height), L 길이(Length)

Copyright © Gilbut, Inc. All rights reserved.

• 3D 입력을 갖는 2D 합성곱

- 입력이 3D 형태임에도 출력 형태가 3D가 아닌 2D 행렬을 취하는 것
- 필터에 대한 길이(L)가 입력 채널의 길이(L)와 같아야 하기 때문
- 입력(W, H, L)에 필터(k, k, L)를 적용하면 출력은 (W, H)
- 필터는 두 방향으로 움직이며, 출력 형태는 2D행렬
- 대표 예시: LeNet-5, VGG

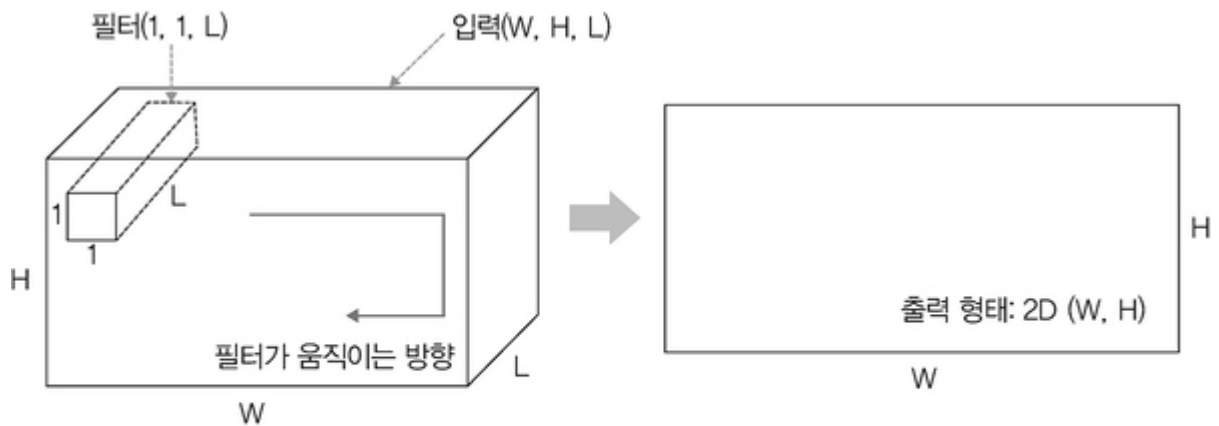


- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: $k \times k$ (높이 $\times$ 너비), L 길이(Length)
- 출력: W 너비(Width), H 높이(Height)

Copyright © Gilbut, Inc. All rights reserved.

- **1×1 합성곱**

- 3D 형태로 입력
- 입력(W, H, L)에 필터(1, 1, L)를 적용하면 출력은 (W, H)
- 채널 수를 조정해 연산량이 감소되는 효과 있음
- **대표 예시: GoogleNet**



- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: 1×1(높이×너비), L 길이(Length)
- 출력: W 너비(Width), H 높이(Height)

Copyright © Gilbut, Inc. All rights reserved.

5.2 합성곱 신경망 맛보기

Note ≡ | fashion_mnist 데이터셋

fashion_mnist 데이터셋은 토치비전(torchvision)에 내장된 예제 데이터로 운동화, 셔츠, 샌들 같은 작은 이미지의 모음이며, 기본 MNIST 데이터셋처럼 열 가지로 분류될 수 있는 28×28 픽셀의 이미지 7만 개로 구성되어 있습니다.

데이터셋을 자세히 살펴보면 훈련 데이터(train_images)는 0에서 255 사이의 값을 갖는 28×28 크기의 넘파이(NumPy) 배열이고, 레이블(정답) 데이터(train_labels)는 0에서 9까지 정수 값을 갖는 배열입니다.

0에서 9까지 정수 값은 이미지(운동화, 셔츠 등)의 클래스를 나타내는 레이블입니다. 각 레이블과 클래스는 다음과 같습니다.

- 0 : T-Shirt
- 1 : Trouser
- 2 : Pullover
- 3 : Dress
- 4 : Coat
- 5 : Sandal
- 6 : Shirt
- 7 : Sneaker
- 8 : Bag
- 9 : Ankle Boot